Applying EfficientNetV2 Variants for Face Verification

Python 3.11 - PyTorch - EfficientNetV2 S/M/L - ArcFace + Hard Pair Mining - LFW Benchmark

SECTION	STATUS	TASK DESCRIPTION	ASSIGNED	DEPENDENCIES	NOTES
Environment Setup	☑	Set up project repo, virtual environment (Python 3.11), install all dependencies (torch, torchvision, facenet-pytorch, timm, scikit-learn, opencv). Verify GPU availability.	**Cuong**	*None*	Run: nvidia-smi. Share requirements.txt with team. Document CUDA version.
	☑	Pull and verify EfficientNetV2 pretrained weights from timm. Confirm S/M/L variants load correctly and output 1280-dim embeddings.	**Cuong**	*venv ready*	Use: timm.create_model('tf_efficientnetv2_s', pretrained=True). Test forward pass with dummy input.
Data Pipeline	☑	Download LFW dataset. Implement mtcnn_preprocess.py using facenet-pytorch MTCNN: detect faces, align, crop, resize to 300x300 / 384x384 / 480x480 for S/M/L. Save processed crops.	**Huy**	*venv ready*	Store crops in data/processed/. Log failed detections. Expected: ~13,000 valid crops.
	☑	Implement dataset.py: LFWPairsDataset class loading the 6,000 verification pairs. Implement dynamic Hard Pair Miner - online mining within each batch to surface Hard Positives and Hard Negatives.	**Huy**	*mtcnn_preprocess.py*	Hard Positive: same person, high distance. Hard Negative: different person, low distance. Use cosine similarity to rank pairs per batch.
Baseline Models	☑	Implement backbone.py: EfficientNetV2 wrapper class supporting S/M/L variants via timm. Remove classifier head, add L2 normalization layer to output 1280-dim unit embeddings.	**Hai**	*Previous*	L2 normalization critical: embedding = F.normalize(features, p=2, dim=1). Makes cosine similarity equivalent to Euclidean.
	☑	Implement siamese.py: Siamese Network processing pairwise inputs via shared backbone weights. Single backbone called twice — not two separate models.	**Hai**	*backbone.py*	Forward: emb_a = backbone(img_a), emb_b = backbone(img_b). Return both embeddings + cosine similarity score.
	☑	Implement losses.py: ContrastiveLoss (Euclidean distance + margin). Train 3 baseline models (EfficientNetV2-S, M, L) using contrastive loss with standard random pair sampling.	**Hai**	*siamese.py + dataset.py*	Margin = 1.0. Log training loss per epoch. Save checkpoints to outputs/checkpoints/baseline_{s,m,l}.pt
Advanced Models	☑	Implement ArcFace loss in losses.py: Additive Angular Margin Loss with scale s=64, margin m=0.5. Optimize angular margin to create tighter intra-class clusters and wider inter-class gaps for lookalikes.	**Linh**	*losses.py baseline*	ArcFace formula: cos(theta + m). Tune margin m on validation split. Tighter clusters directly reduces False Accepts for lookalikes.
	☑	Implement train_arcface.py: Train 3 advanced models (S/M/L) using ArcFace loss + Hard-Mined pairs from dataset.py. 6 total training runs: 3 baseline + 3 ArcFace.	**Linh**	*ArcFace loss + hard miner*	Hard mining + ArcFace is the core research contribution. Log: loss, EER per epoch. Save to outputs/checkpoints/arcface_{s,m,l}.pt

Cuong	8
Hai	3
Huy	2
Linh	4

	☑	Implement configs/ YAML files (6 total). Each config specifies: variant, loss_type, margin, scale, batch_size, learning_rate, input_size, mining_strategy.	**Linh**	*train scripts ready*	Centralize all hyperparameters. No hardcoded values in training scripts. Use PyYAML to load.
Evaluation	☐	Implement metrics.py: FAR, FRR, EER calculation. Plot ROC curves and FAR/FRR vs threshold curves for all 6 models.	**Cuong**	*All 6 models trained*	EER = threshold where FAR == FRR. Target: EER < 5% on LFW. Latency: time per pair inference on CPU and GPU.
	☐	Implement evaluate_lfw.py: run all 6 trained models on LFW 6,000-pair standard protocol. Generate side-by-side FAR/FRR/EER and latency comparison table (S vs M vs L, baseline vs ArcFace).	**Cuong**	*metrics.py + all checkpoints*	Table must show: model, loss_type, EER, FAR@FRR1%, latency_ms. Export to outputs/metrics/comparison_table.csv
	☐	Implement error_analysis.py: Visual Error Analysis. Isolate False Accept pairs (lookalikes that passed) and False Reject pairs (same person wrongly blocked). Visualize side-by-side with similarity scores.	**Cuong**	*evaluate_lfw.py*	Output to outputs/error_cases/. Show: pair images, predicted score, threshold, correct label. Proves ArcFace effectiveness over baseline.
Demo	☐	Implement demo/verify.py: live camera verification demo. Enroll face from webcam → extract embedding → capture test face → cosine similarity → match/reject decision with threshold display.	**Linh**	*Best model checkpoint*	Use best performing model (lowest EER). Display: similarity score, threshold, MATCH/REJECT. Maps to exam authentication scenario.
Finalization	☐	Write README.md: installation, dataset setup, how to run main.py with examples, expected outputs. Write setup.sh to automate venv + dependency installation.	**Cuong**	*All features stable*	README must let a new user run full pipeline within 15 minutes of cloning. Include: EER table, error case examples.
	☐	Code review across all modules. Resolve TODOs. Ensure no hardcoded paths. Final repo cleanup, tag release v1.0, confirm all imports work from fresh venv.	**Cuong**	*All code stable*	Check: .gitignore covers outputs/checkpoints/, requirements.txt is complete, all 6 configs load correctly.
	☐	End-to-end integration test: run full pipeline from raw LFW → MTCNN preprocessing → training (1 epoch) → evaluation → error analysis → demo. Document any failures.	**Cuong**	*Full pipeline complete*	Test on small subset first (200 pairs). Verify checkpoints saved, results CSV exported, error cases visualized.